

# CYBERSECURITY INTERNSHIP REPORT

Redynox — Cyber Security Internship Program

---

**Intern Name:** Sumit Jaiswal

**Intern ID:** RDXINTSUMIW89MD

**Role:** Cyber Security Intern

**Duration:** 6 August 2026 – 5 September 2026 (1 Month)

**Mode:** Remote

**Reporting Manager:** Ankit Mathur, Internship Supervisor/Lead

**Academic Background:** B.Tech CSE (Cybersecurity Specialization), Final Year

**Report Date:** 5 September 2026

## Task 2: Introduction to Web Application Security

- **Objective:** To identify and understand common web application vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF) using OWASP ZAP and WebGoat.
- **Tools & Methods Used:** OWASP ZAP v2.17.0, WebGoat (local vulnerable web application, WebGoat 2025.3), WebWolf, Firefox configured with ZAP proxy (localhost:8080).

### Steps Taken

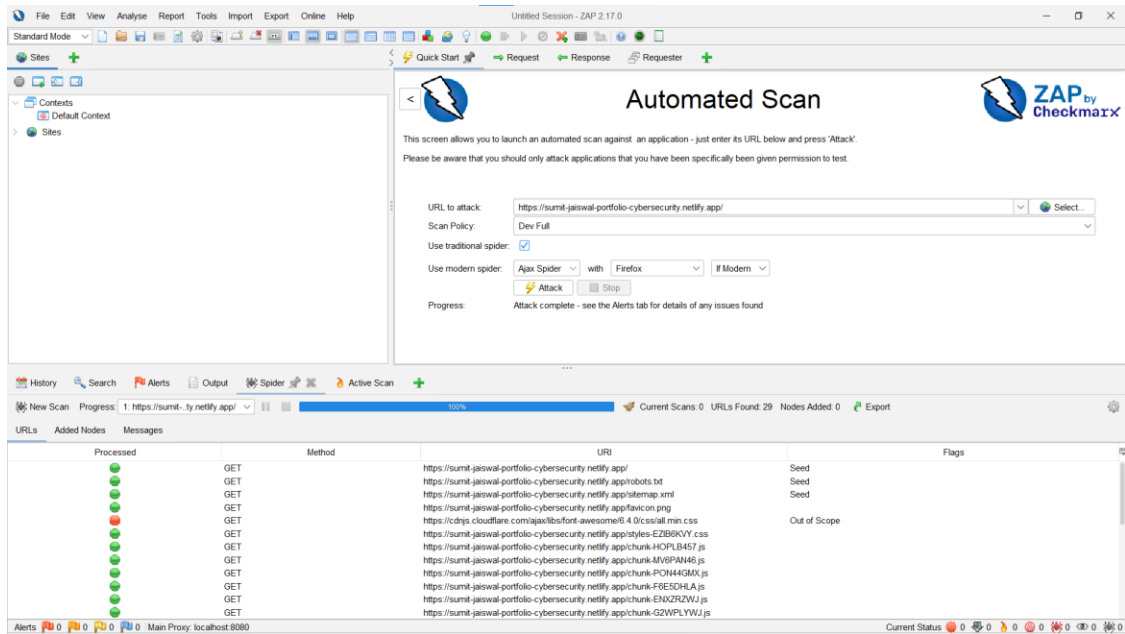
1. Installed and started WebGoat locally on my system for safe, legal vulnerability testing.
2. Logged into WebGoat and explored the lesson menu, which is organised by OWASP Top 10 categories (Injection, XSS, CSRF, Broken Access Control, etc.).
3. Set up OWASP ZAP and ran an Automated Scan against my own portfolio site (<https://sumit-jaiswal-portfolio-cybersecurity.netlify.app/>) using the "Dev Full" scan policy.
4. Reviewed the ZAP History and Sites tabs to see every request the scanner made, and which external resources were correctly marked "Out of Scope."
5. Repeated the automated scan against a second test application to compare results and confirm I understood how to read ZAP's scan progress and history correctly.
6. Worked through the SQL Injection lessons in WebGoat, attempting to log in as the user "Tom" using classic SQL Injection payloads (e.g. Tom'--) to bypass the password check.
7. Worked through the Cross-Site Scripting (XSS) lessons in WebGoat, injecting a script payload into an input field that the application reflected back without sanitizing it, and completed a second exercise showing how an attacker could read a user's session cookie through XSS.
8. Attempted the CSRF lessons in WebGoat, including using WebWolf (WebGoat's built-in "attacker" tool) to simulate a request coming from an external site rather than from WebGoat itself.

### Setup: Installing and Exploring WebGoat

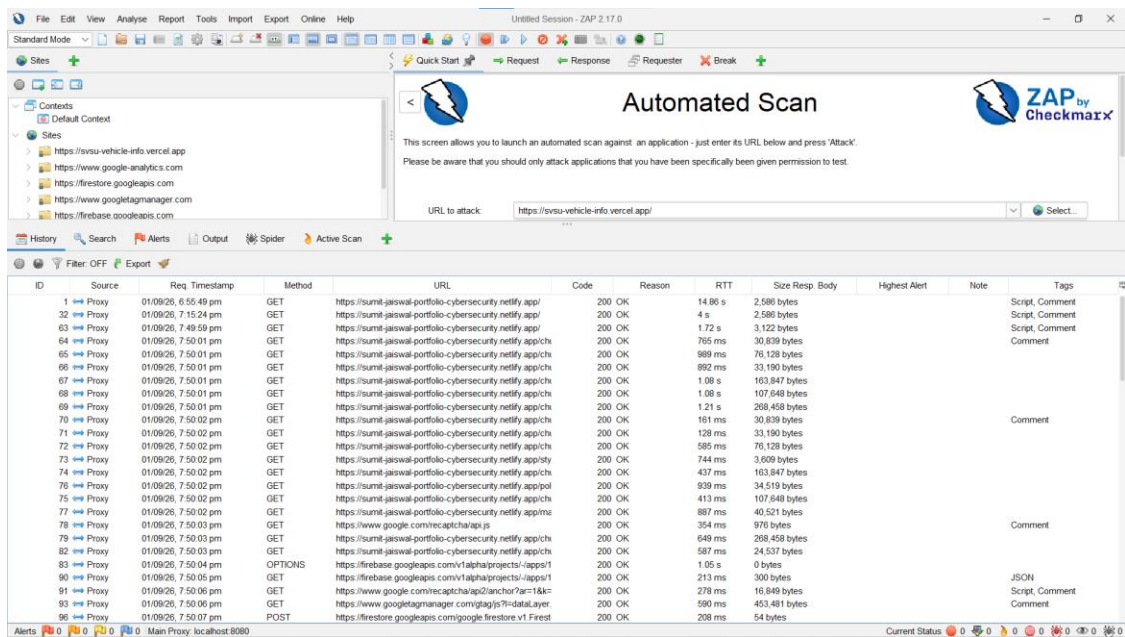
I started WebGoat from the command line using its jar file. Once the embedded Tomcat server was up and running on port 9090, I logged in and was shown the lesson menu, organised by OWASP Top 10 vulnerability categories — this is where all the SQL Injection, XSS, and CSRF exercises live.

*Figure 1: WebGoat starting up from the command line (embedded Tomcat server on port 9090).*





**Figure 4: ZAP Sites tree and History tab, showing the full list of requests captured during the scan.**



**Figure 5: Detailed ZAP History view of a scanned application, including login, admin, and API routes discovered by the spider.**

The screenshot shows the ZAP Automated Scan interface. The main window displays the title "Automated Scan" and a brief instruction: "This screen allows you to launch an automated scan against an application - just enter its URL below and press 'Attack'." Below this, the "URL to attack:" field contains "https://vsu-vehicle-info.vercel.app/".

The bottom section of the interface shows a table of scan results. The table has columns: ID, Source, Req. Timestamp, Method, URL, Code, Reason, RTT, Size Resp. Body, Highest Alert, Note, and Tags. The results are as follows:

| ID  | Source | Req. Timestamp       | Method | URL                                                            | Code | Reason                | RTT     | Size Resp. Body | Highest Alert | Note | Tags                  |
|-----|--------|----------------------|--------|----------------------------------------------------------------|------|-----------------------|---------|-----------------|---------------|------|-----------------------|
| 152 | Proxy  | 01/09/20, 7:51:34 pm | GET    | https://vsu-vehicle-info.vercel.app/admin?_rsc=1r34m           | 307  | Temporary Redirect    | 156 ms  | 15 bytes        |               |      |                       |
| 153 | Proxy  | 01/09/20, 7:51:34 pm | GET    | https://vsu-vehicle-info.vercel.app/api/profile                | 500  | Internal Server Error | 342 ms  | 9,205 bytes     |               |      | Form, Script, Comment |
| 154 | Proxy  | 01/09/20, 7:51:34 pm | GET    | https://vsu-vehicle-info.vercel.app/terms?_rsc=1r34m           | 200  | OK                    | 329 ms  | 313 bytes       |               |      |                       |
| 155 | Proxy  | 01/09/20, 7:51:34 pm | GET    | https://vsu-vehicle-info.vercel.app/visitor-entry?_rsc=1r3     | 200  | OK                    | 319 ms  | 321 bytes       |               |      |                       |
| 156 | Proxy  | 01/09/20, 7:51:34 pm | GET    | https://vsu-vehicle-info.vercel.app/api/dashboard              | 307  | Temporary Redirect    | 133 ms  | 15 bytes        |               |      |                       |
| 157 | Proxy  | 01/09/20, 7:51:34 pm | GET    | https://vsu-vehicle-info.vercel.app/login                      | 200  | OK                    | 335 ms  | 313 bytes       |               |      |                       |
| 158 | Proxy  | 01/09/20, 7:51:34 pm | GET    | https://vsu-vehicle-info.vercel.app/terms?_rsc=1r34m           | 200  | OK                    | 327 ms  | 1,267 bytes     |               |      |                       |
| 159 | Proxy  | 01/09/20, 7:51:35 pm | GET    | https://vsu-vehicle-info.vercel.app/terms?_rsc=1r34m           | 200  | OK                    | 354 ms  | 4,889 bytes     |               |      |                       |
| 160 | Proxy  | 01/09/20, 7:51:34 pm | GET    | https://vsu-vehicle-info.vercel.app/terms?_rsc=1r34m           | 200  | OK                    | 477 ms  | 525 bytes       |               |      |                       |
| 161 | Proxy  | 01/09/20, 7:51:35 pm | GET    | https://vsu-vehicle-info.vercel.app/login                      | 200  | OK                    | 336 ms  | 9,799 bytes     |               |      | Script, Comment       |
| 162 | Proxy  | 01/09/20, 7:51:35 pm | GET    | https://vsu-vehicle-info.vercel.app/login?_rsc=1r34m           | 200  | OK                    | 486 ms  | 1,267 bytes     |               |      |                       |
| 163 | Proxy  | 01/09/20, 7:51:35 pm | GET    | https://vsu-vehicle-info.vercel.app/visitor-entry?_rsc=1r3     | 200  | OK                    | 301 ms  | 525 bytes       |               |      |                       |
| 164 | Proxy  | 01/09/20, 7:51:35 pm | GET    | https://vsu-vehicle-info.vercel.app/login?_rsc=1r34m           | 200  | OK                    | 331 ms  | 1,024 bytes     |               |      |                       |
| 165 | Proxy  | 01/09/20, 7:51:35 pm | GET    | https://vsu-vehicle-info.vercel.app/login?_rsc=1r34m           | 200  | OK                    | 361 ms  | 525 bytes       |               |      |                       |
| 166 | Proxy  | 01/09/20, 7:51:35 pm | GET    | https://vsu-vehicle-info.vercel.app/visitor-entry?_rsc=1r3     | 200  | OK                    | 359 ms  | 1,267 bytes     |               |      |                       |
| 167 | Proxy  | 01/09/20, 7:51:35 pm | GET    | https://vsu-vehicle-info.vercel.app/_next/static/chunks/0      | 200  | OK                    | 60 ms   | 4,662 bytes     |               |      |                       |
| 168 | Proxy  | 01/09/20, 7:51:35 pm | GET    | https://vsu-vehicle-info.vercel.app/visitor-entry?_rsc=1r3     | 200  | OK                    | 314 ms  | 1,024 bytes     |               |      |                       |
| 169 | Proxy  | 01/09/20, 7:51:35 pm | GET    | https://vsu-vehicle-info.vercel.app/_next/static/chunks/0      | 200  | OK                    | 343 ms  | 19,459 bytes    |               |      |                       |
| 170 | Proxy  | 01/09/20, 7:51:23 pm | GET    | https://firestore.googleapis.com/google.firestore.v1.Firestore | 200  | OK                    | 14.88 s | 158 bytes       |               |      |                       |
| 171 | Proxy  | 01/09/20, 7:51:35 pm | POST   | https://www.google.com/recaptcha/api/ch?c=YOUR_R               | 400  | Bad Request           | 256 ms  | 1,684 bytes     |               |      |                       |
| 172 | Proxy  | 01/09/20, 7:51:35 pm | GET    | https://vsu-vehicle-info.vercel.app/_next/static/chunks/0      | 200  | OK                    | 68 ms   | 4,662 bytes     |               |      |                       |
| 173 | Proxy  | 01/09/20, 7:51:35 pm | GET    | https://vsu-vehicle-info.vercel.app/_next/static/chunks/0      | 200  | OK                    | 78 ms   | 15,018 bytes    |               |      |                       |
| 174 | Proxy  | 01/09/20, 7:51:35 pm | GET    | https://vsu-vehicle-info.vercel.app/_next/static/chunks/0      | 200  | OK                    | 74 ms   | 19,459 bytes    |               |      |                       |
| 175 | Proxy  | 01/09/20, 7:51:54 pm | GET    | https://www.google.com/complete/search?client=firefox          | 200  | OK                    | 251 ms  | 727 bytes       |               |      |                       |
| 177 | Proxy  | 01/09/20, 7:52:33 pm | GET    | https://vsu-vehicle-info.vercel.app/                           | 200  | OK                    | 474 ms  | 9,315 bytes     |               |      |                       |

**Figure 6: Repeating the automated scan against a second test application using the "QA Full 2" scan policy, to confirm consistent results.**

The screenshot shows the ZAP Automated Scan interface. The main window displays the title "Automated Scan" and a brief instruction: "This screen allows you to launch an automated scan against an application - just enter its URL below and press 'Attack'." Below this, the "URL to attack:" field contains "https://lovetwithyou.vercel.app/".

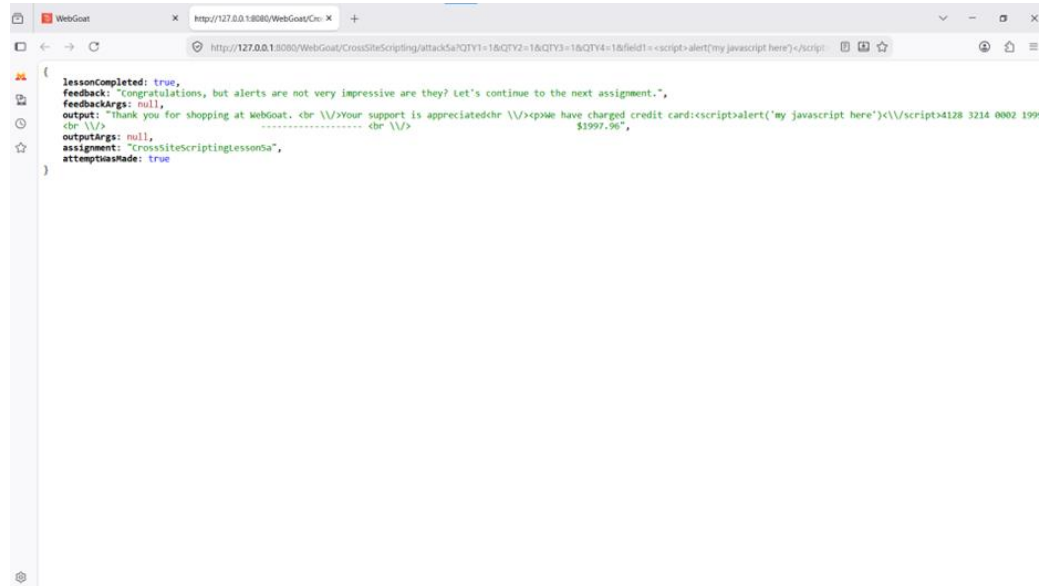
The bottom section of the interface shows a table of scan results. The table has columns: Processed, Method, URL, and Flags. The results are as follows:

| Processed | Method                                                                         | URL | Flags |
|-----------|--------------------------------------------------------------------------------|-----|-------|
| GET       | https://lovetwithyou.vercel.app/_next/static/chunks/turbopack-0a71g13pg_z3r.js |     |       |
| GET       | https://lovetwithyou.vercel.app/_next/static/chunks/1m_5d1_r20ee.js            |     |       |
| GET       | https://lovetwithyou.vercel.app/_next/static/chunks/0ray9d0t_6th.js            |     |       |
| GET       | https://lovetwithyou.vercel.app/_next/static/chunks/05yyfc_0poj.js             |     |       |
| GET       | https://lovetwithyou.vercel.app/_next/static/chunks/2b558r_g0h1.js             |     |       |
| GET       | https://lovetwithyou.vercel.app/_next/static/chunks/17k_rnf_5ymk6.js           |     |       |
| GET       | https://lovetwithyou.vercel.app/_next/static/chunks/07h6fz_b42v.js             |     |       |
| GET       | https://lovetwithyou.vercel.app/_next/static/chunks/0b6h1-1b0k.js              |     |       |
| GET       | https://lovetwithyou.vercel.app/_next/static/chunks/05g_7bx1b0u.js             |     |       |
| GET       | https://lovetwithyou.vercel.app/_next/static/chunks/447fbybucd0.js             |     |       |
| GET       | https://lovetwithyou.vercel.app/_next/static/chunks/0cz100m5g_q7.js            |     |       |
| GET       | https://lovetwithyou.vercel.app/_next/static/chunks/069_y58-maoi.js            |     |       |

## Manual Exploitation — Cross-Site Scripting (XSS)

In the WebGoat XSS lessons, I found an input field whose value was reflected back into the page without being sanitised. Submitting a payload such as `<script>alert('my javascript here')</script>` caused WebGoat to confirm the lesson was completed, and the raw JSON response showed my script sitting unescaped inside the page's output — proof that the application was not encoding user input before rendering it. In a follow-up exercise, I also confirmed that the same session cookie was accessible via JavaScript on two different tabs, which is exactly what an attacker's injected script could steal and reuse to hijack a logged-in session.

**Figure 7: XSS payload reflected unescaped in the application's JSON response, with lessonCompleted: true confirming the exploit worked.**



**Figure 8: WebGoat confirming the XSS cookie-theft exercise was completed successfully.**

## What is XSS?

Cross-Site Scripting (also known as XSS) is a vulnerability/ flaw that combines the allowance of HTML/script tags as input that renders into a browser without encoding or sanitization.

Cross-Site Scripting (XSS) is the most prevalent and pernicious web application security issue

While there is a simple well-known defense for this attack, there are still many instances on the web. Coverage of fixes also tends to be a problem in terms of fixing it. We will talk more about the defense in a little bit.

## XSS has significant impact

Especially as 'Rich Internet Applications' are more and more commonplace, privileged function calls linked to via JavaScript may be compromised. And if not adequately protected, sensitive data (such as your authentication cookies) can be stolen and used for someone else's purpose.

### Quick examples:

- From the JavaScript console in the developer tools of the browser (Chrome, Firefox)

```
alert("XSS Test");
alert(document.cookie);
```

- Any data field returned to the client is potentially injectable

```
<script>alert("XSS Test")</script>
```

## Try It! Using Chrome or Firefox

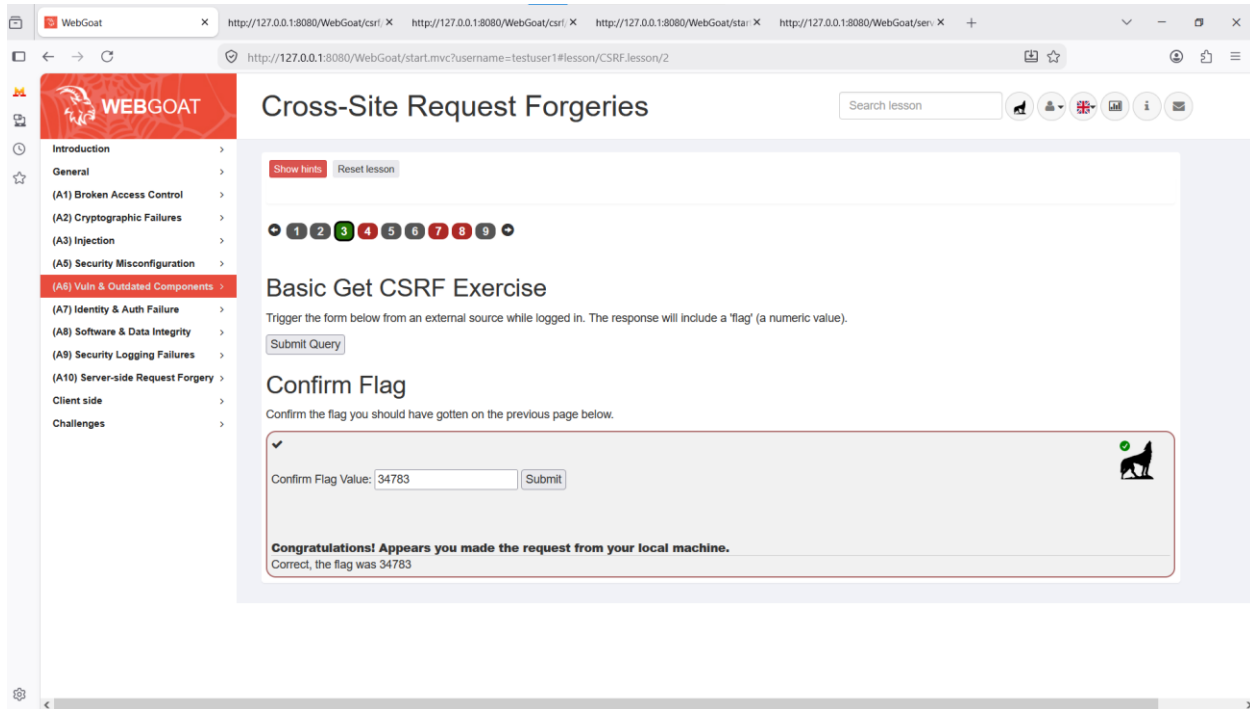
- Open a second tab and use the same URL as this page you are currently on (or any URL within this instance of WebGoat).
- On the second tab, open the JavaScript console in the developer tools and type: `alert(document.cookie);`.
- The cookies should be the same on each tab.

☒ The cookies are the same on each tab

**Congratulations. You have successfully completed the assignment.**

## Manual Exploitation — SQL Injection and CSRF

For SQL Injection, I worked through WebGoat's "Log in as Tom" challenge, which asks the learner to combine everything taught in the earlier SQL Injection lessons to bypass the login form without knowing Tom's real password. For CSRF, I worked through the "Basic Get CSRF" exercise, which requires triggering a request from an external origin (using WebGoat's built-in WebWolf tool to host a fake attacker page) rather than from within WebGoat itself, so that the flag is only returned when the request genuinely looks like it came from a third-party site.



**SQL Injection section update karo:** Us section me yeh mention kar do ki "Log in as Tom" challenge ke dauran kai payloads try kiye gaye jaise Tom' --, Tom' OR '1'=1, Tom' /\*, aur Second-Order SQL Injection ke liye registration trick (tom' AND '1'=1 --), jisse ultimately database query behavior samajh kar vulnerability confirm ki gayi. But not working I think due to new update

## Suggested Mitigation Steps

- Use parameterized queries / prepared statements to prevent SQL Injection, instead of building SQL strings from raw user input.
- Sanitize and encode all user input before reflecting it back into a page, to prevent XSS.
- Implement anti-CSRF tokens on all state-changing forms and requests, and reject requests that do not include a valid token.
- Add security headers such as Content-Security-Policy and X-Frame-Options to reduce the impact of any XSS or clickjacking attempts.

## Challenges Faced

- My first SQL Injection attempts failed because I only manipulated the username field; the password condition in the underlying query was still being evaluated. Using a comment sequence (--) to cut off the rest of the query was the key idea I was missing initially.
- A DOM-Based XSS lesson required digging through the application's JavaScript source to find a hidden route — this was too advanced for a beginner exercise, so I focused on the simpler reflected-XSS lessons instead, which still demonstrated the vulnerability clearly.

- My first CSRF attempt failed because I triggered the request directly from within WebGoat, so it was correctly detected as coming from "the original host" rather than an external attacker — this is what led me to use WebWolf to simulate a genuinely external request.

## Results & Outcomes

Through this task I learned how an automated scanner like OWASP ZAP maps out an application and flags potential issues, and — more importantly — how to manually confirm those vulnerabilities are real by exploiting them myself in a safe, legal environment (WebGoat). Successfully exploiting the XSS vulnerability, and understanding exactly why my first SQL Injection and CSRF attempts failed, gave me a much clearer picture of how these three OWASP Top 10 vulnerabilities actually work and why the mitigations listed above are necessary in real applications.